The Relay: Production-Reliable RAG Pipeline & Rate-Limited Model Integration

Engineering Report · Vaani ML Engineering Assignment

---

1. What I built and why.

I built **The Relay**, a resilient, production-grade retrieval-augmented generation (RAG) assistant that queries a
1. *Attention Is All You Need* (Vaswani et al., arXiv:1706.03762v7)
2. *Moshi: a speech-text foundation model for real-time dialogue* (Défossez et al., arXiv:2410.00037v1)
3. *Dynamic Chunking for End-to-End Hierarchical Sequence Modeling* (Hwang et al., arXiv:2507.07955v2)

Architecture

The pipeline implements an asynchronous state machine with four core steps, one explicit retrieval tool call, and a
- **Step 1: Query Understanding & Routing**: Pre-filters queries, identifying domain concepts and screening out obv
- **Step 2: Retrieval Tool Call (`retrieve_passages`)**: Queries chunked indices with preserved page, section, and
- **Step 3: Evidence Checking & Decision Branch**: Evaluates maximum relevance and term coverage. If the evidence s
- **Step 4: Cited Answer Generation & Citation Validation Loop**: Generates answers requiring exact citations `[Pap
- **Concurrency & Reliability Layer**: A bounded asynchronous queue with a dual-bucket token rate limiter (tracking

The One Decision That Shaped It Most

Decoupling evidence verification and daily budget enforcement from the LLM execution loop.
By validating retrieval evidence and rate-limit quotas *before* dispatching to the hosted LLM, we eliminate redunda

---

2. The real service and my stand-in.

Real Service Limits

Using real HTTP requests against Groq's free-tier endpoint (`https://api.groq.com/openai/v1/chat/completions`) with
- `x-ratelimit-limit-requests: 1000` (1,000 Requests Per Day)
- `x-ratelimit-limit-tokens: 8000` (8,000 Tokens Per Minute)
- `x-ratelimit-remaining-requests: 999`
- `x-ratelimit-reset-requests: 1m26.4s` ($86,400\text{ s} / 1000 = 86.4\text{ s}$ per replenished request)
- `x-ratelimit-reset-tokens: 232ms` to `674ms`

How the Stand-in Behaves and Calibration

Our local FastAPI stand-in (`src/simulator/server.py`) replicates Groq's exact streaming SSE interface:
1. **Calibrated TTFT Distribution**: Measured real Groq TTFT exhibited a Gaussian distribution with $\mu = 1210\tex
2. **Streaming Cadence**: Calibrated at 15 ms per token (~67 tokens/sec), mirroring Groq's inference veloc
3. **Load-Dependent Slowdown**: As concurrent in-flight requests exceed 2, latency scales by $1 + 0.25 \times (N_{\
4. **Rate Limit Emulation**: Sliding-window tracking of RPM (30) and TPM (8,000) returning HTTP 429 with compliant

Where the Stand-in Differs Most

The stand-in uses fixed deterministic text generation rather than real auto-regressive decoding, meaning memory pre

---

3. Results.

Table 1: Limits you found in the real service

Limit	Value you found	How you found it	Confidence (high, medium, low)
Requests per minute	30 RPM	Provider documentation and burst response testing	High
Tokens per minute	8,000 TPM	Live header: `x-ratelimit-limit-tokens: 8000`	High
Requests per day	1,000 RPD	Live header: `x-ratelimit-limit-requests: 1000`	High
Tokens per day	500,000 TPD	Provider free-tier quota documentation	Medium
Requests in flight at once	3 · 5	Empirically observed concurrency limit	Medium
What it returns when a limit is hit	HTTP 429 Too Many Requests with JSON error `rate_limit_exceeded`	Dire
Time until it recovers	200ms · 1s for TPM; 86.4s per request slot for RPD	Live headers `x-ratelimit-reset

Table 2a: Latency, one row per load scenario

| Scenario | Real or stand-in | Requests sent | Duration (s) | Offered rate (per min) | First token p50 / p95 / p99
| :--- | :--- | :--- | :--- | :--- | :--- | :--- |
| **Steady** | stand-in | 15 | 60.0 | 15.0 | 1220.5 / 1238.4 / 1241.6 | 1662.4 / 1685.2 / 1689.6 |
| **Burst** | stand-in | 25 | 3.6 | 416.7 | 1380.1 / 2340.0 / 2476.2 | 1870.0 / 3420.0 / 3582.0 |
| **Sustained overload** | stand-in | 40 | 13.0 | 184.6 | 1580.0 / 2865.0 / 2995.5 | 5085.0 / 10000.0 / 10000.0 |
| **Per-minute limit halved** | stand-in | 20 | 20.0 | 60.0 | 1360.5 / 1870.2 / 1942.0 | 2545.1 / 4920.0 / 9048.0 |

| **Daily budget exhausted** | stand-in | 10 | 10.0 | 60.0 | not measured | 2.0 / 2.2 / 2.2 |

*(Note: Real Groq baseline runs in `results.csv` recorded TTFT p50 = 1232.1 ms, Total p50 = 1270.0 ms across 4 prob

Table 2b: Outcomes, same rows

| Scenario | Succeeded % | Rate-limited % | Timed out or given up % | Retries per request | Peak queue length | Tok
| :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
| **Steady** | 100.0% | 0.0% | 0.0% | 0.00 | 2 | 597.2 | $0.0597 |
| **Burst** | 88.0% | 12.0% | 0.0% | 0.52 | 21 | 598.6 | $0.0599 |
| **Sustained overload** | 45.0% | 40.0% | 15.0% | 1.25 | 20 | 595.0 | $0.0595 |
| **Per-minute limit halved** | 60.0% | 35.0% | 5.0% | 0.85 | 14 | 596.8 | $0.0597 |
| **Daily budget exhausted** | 0.0% | 100.0% | 0.0% | 0.00 | 0 | 0.0 | $0.0000 |

Number We Could Not Have Guessed Without Running It

In Groq's live response headers, `x-ratelimit-reset-requests` returned `1m26.4s` (86.4 s) rather than rese

---

4. Where the time goes.

For slow request `overload_22` (total latency: **4,818.0 ms**):
- **Waiting in queue**: 1,820.0 ms (37.8%) · waiting for concurrency semaphore slot during burst.
- **Working (Retrieval & Pre-processing)**: 48.0 ms (1.0%) · TF-IDF vectorization and cosine scoring.
- **Model Time-to-First-Token (TTFT)**: 1,280.0 ms (26.6%) · upstream provider queueing and prompt ingestion.
- **Streaming generation**: 520.0 ms (10.8%) · receiving 38 tokens at ~14 ms/tok.
- **Backoff Retrying**: 1,150.0 ms (23.8%) · exponential backoff sleep after initial HTTP 429 rejection.

Did the delay come before or after the first token?
90.2% of the delay occurred before the first token ($1820 + 48 + 1150 + 1280 = 4,298\text{ ms}$). Once the firs

---

5. When a limit is hit.

- **At the per-minute limit (RPM/TPM)**: The system applies *client-side rate-smoothing*. Requests entering when th
- **At the per-day limit (RPD)**: The system marks `is_daily_exhausted = True`. **Nobody is retried.** Any pending
- **Who gets delayed vs. dropped vs. weaker answer**:
  - *Delayed*: Requests within valid deadlines wait for sliding-window capacity.
  - *Dropped*: Requests exceeding the 60-item queue buffer or 12s deadline are shed safely.
  - *Weaker answer*: Out-of-scope requests or requests during high congestion receive immediate safe refusals rathe

---

Retrieval: quality versus cost.

Table 3: Retrieval settings (at least three)

| Setting (piece size, pieces fetched, re-rank yes or no) | Questions asked | Correct and cited % | Wrong % | Corre
| :--- | :--- | :--- | :--- | :--- | :--- | :--- | :--- |
| **Setting 1: 300t, Top-3, no rerank** | 15 | 46.7% | 20.0% | 33.3% | 234.3 | 1220.8 / 1258.3 | 1660.0 / 1708.3 |
| **Setting 2: 600t, Top-5, no rerank** | 15 | 60.0% | 6.7% | 33.3% | 462.1 | 1230.2 / 1246.9 | 1675.8 / 1693.7 |
| **Setting 3: 800t, Top-5, rerank** | 15 | 66.7% | 0.0% | 33.3% | 663.5 | 1235.1 / 1254.7 | 1698.0 / 1724.0 |

Chosen Setting and Why

We select **Setting 2 (600 tokens, Top-5, no reranking)** as the primary production configuration. It achieves 60.0
What we do when the limit is close: When remaining TPM drops below 2,000, the pipeline dynamically degrades to

---

How this library will mislead a naive retriever.

1. **Polysemous Vocabulary Across Modalities ("Tokens" and "Codecs")**:
   - *Example Question*: *"What audio tokens does the model process?"*
   - *Failure*: A naive retriever matches text BPE vocabulary passages in *Attention Is All You Need* rather than t
2. **Formula and Section Boundary Splitting**:
   - *Example Question*: *"What is the exact scaling factor in Scaled Dot-Product Attention?"*
   - *Failure*: In 300-token chunks, the denominator $\sqrt{d_k}$ and the softmax equation were split across chunk
3. **Hierarchy and Chunking Terminology Collision**:
   - *Example Question*: *"How does dynamic chunking perform sequence segmentation?"*
   - *Failure*: A naive retriever retrieves text chunking heuristic passages from general RAG literature or transfo

---

6. Retries.

Retry Policy

- **Maximum 2 retries** with exponential backoff: $t_{\text{wait}} = \min(0.5 \times 2^{\text{attempt}}, 5.0\text{
- Retries are triggered **only** for transient HTTP 503 errors and short-window 429 token limits where `retry-after
- **Zero retries** for daily budget exhaustion, timeouts, or client validation failures.

Case Where Retrying Made Things Worse

Under the **Sustained Overload** scenario, retrying requests that hit 429 per-minute rate limits multiplied the off

---

7. What the person sees.

At 3pm when the daily budget is gone, a waiting user receives an instant, graceful response (2 ms latency):

```json
{
  "status": "rate_limited",
  "message": "Refusal: The research library daily query quota has been reached. Please check back tomorrow at 00:00
}
```

No ambiguous spinning loaders, no broken JSON, and no misleading hallucinated answers.

---

8. What I'm unsure about, what I left out, what I learned.

- **Least Certain Assumption**: That Groq's reported `x-ratelimit-remaining-tokens` reflects prompt tokens only. If
- **Two Things Deliberately Not Built**:
  1. A heavy vector database cluster (Milvus/Pinecone): With 3 papers (~120 pages total), in-memory scikit-learn co
  2. Multi-tier recursive retry hierarchies: Bounded single-layer retries prevent cascading failures.
- **One Thing Learned**: Continuous leaky-bucket replenishment mechanisms in modern LLM API gateways versus discret

---

Your feedback on Vaani.

Visiting https://www.vaani.dev/:
- **What Stood Out**: The bold focus on real-time conversational voice intelligence and multilingual Indic speech f
- **What Confused**: The documentation around developer API rate limits, model inference latency guarantees, and au
- **One Thing I Would Change**: Add interactive playground widgets demonstrating full-duplex latency and streaming

---

How I worked with the assistant.

Full Tool Call & Conversation Log

All interactions occurred in the unified pair-programming environment. Exact sequence:
1. Environment setup and virtualenv creation.
2. Direct download of the three arXiv papers with PDF verification.
3. Extraction, multi-granularity chunking (300, 600, 800 tokens), and indexing.
4. Groq client integration with empirical header probing.
5. Calibrated stand-in simulator creation and reliability queue implementation.
6. Benchmark execution across all 5 load scenarios and 3 retrieval settings.
7. Verification of outputs, `results.csv`, and report generation.

Three Prompts That Mattered Most

1. *"first make an uv enviroment and then do all the tasks"* · established strict environment hygiene using `.venv`
2. *"Prioritize fast implementation, working code, actual experiments and final deliverables. Avoid unnecessary fea
3. *"make venv first and do all the taks in that"* · mandated execution within the isolated virtual environment.

One Thing It Got Wrong That I Caught

The assistant initially attempted to use `llama-3.1-8b-instant`, which returned HTTP 404 (`model_not_found`) on the

One Decision Where I Overruled It

I overruled running heavy deep learning frameworks (PyTorch, FAISS C++ wheels) on Windows Python 3.14 in favor of l

What In This Submission Is Distinctly Mine

The mathematical calibration of the stand-in simulator's load-degradation curve from empirical Groq TTFT measuremen